CS780 Capstone Project: OBELIX: the Warehouse Robot

Krishanu Ray
241110037
krishanur24@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This report addresses the OBELIX[1] warehouse robot control problem, where a mobile robot with 18-bit sensor observations must locate, attach to, and push a box to the arena boundary across increasing difficulty levels. I have implemented and experimented with various algorithms across families such as Q-Learning, DDQN, D3QN, and PPO, of which only value-based methods showed promising results. Key innovations include reward clipping, forward biasing, attachment bit, prioritised experience replay and recurrent networks, resulting in strong performance across static, blinking, and moving-box settings.

1 Competition Result

Codabench Username: k_241110037
Final leaderboard rank (Test Phase): 2
Leaderboard rank – Level 1 (static): 15
Leaderboard rank – Level 2 (blinking): 1
Leaderboard rank – Level 3 (moving + blinking): 3

Number of valid submissions made (Development(Level 1 , Level 2, Level 3) and Test Test):

Phase	Number of Submissions
Development Level 1	8
Development Level 2	8
Development Level 3	8
Test	6

2 Introduction and Problem Description

This capstone studies a reinforcement learning agent in the OBELIX environment, a challenging control task where a robot must learn reliable behaviour from limited sensor observations. The core task is to **find** a grey box, **attach** to it, **push** it toward the arena boundary, and recover if it becomes wedged. The robot operates in a 2D arena with five discrete actions and receives only an 18-bit sensor observation rather than the full environment state.

The environment includes three difficulty levels: **difficulty 0** with a static box, **difficulty 2** with a blinking box, and **difficulty 3** with a moving and blinking box. An optional wall obstacle can also be added, making navigation and pushing harder. This setting is a **Partially Observable Markov Decision Process (POMDP)** since the agent must act using incomplete local sensor readings without direct

access to the true state, requiring it to infer hidden information and generalize across changing conditions.

The high-level objective is to **maximize mean cumulative reward across difficulty levels while remaining robust to wall obstacles**. My approach first trains specialised models for individual difficulty levels, then incorporates their key features into a single unified agent that generalises across all settings.

3 Approach and Model Evolution

Please find the YouTube link for demonstration and the GitHub repository link below:

- **Video demonstration:** `https://youtu.be/8tUFtj6AsjU`
- **Code repository:** `https://github.com/nighteagle420/DRL-Capstone-Submissions`

3.1 Phase 1, Week 1: Random Exploration Stage

My first submission was the default sample zip, scoring $-\mathbf{17128.10}$ [Submission 1 (616488) – Sample Submission]. Training the default DDQN without walls scored $-\mathbf{379036.50}$ [Submission 2 (625776) – Default DDQN], improving to $-\mathbf{215282.10}$ with more episodes [Submission 3 (626163) – Default DDQN with more episodes]. I then tried an encoder-LSTM DDQN, but passed experiences individually rather than as sequences, scoring $-\mathbf{29248.90}$ [Submission 4 (626400) – DDQN with LSTM]. Despite the flaw, the improved score suggested recurrent approaches were promising. A Dueling architecture scored $-\mathbf{102740.80}$ [Submission 5 (626946) – Basic D3QN].

Key takeaways: Training without walls does not transfer to wall environments. Iterating on a focused set of algorithms is more productive than trying many different ones. Based on this, I concluded that modifications of DDQN were the most promising direction.

3.2 Phase 1, Week 2: Focused Experimenting

All training used difficulty 0 with walls ON. A tabular Q-Lambda ($\lambda = 0.7$, 10K episodes) scored $-\mathbf{1997.40}$ [Submission 1 (640618) – Q Lambda]. A basic DQN with aggressive epsilon decay scored $-\mathbf{1989.30}$ [Submission 2 (641482) – DQN with aggressive exploration]. Inspired by the original OBELIX paper, I augmented observations with an attachment bit and trained a separate AttachNet to predict it at evaluation time. This scored $-\mathbf{1972.70}$ [Submission 3 (643007) – Augmented DDQN]. A basic PPO scored $-\mathbf{1980.70}$ [Submission 4 (644285) – Basic PPO] regardless of tuning.

Key takeaways: Tabular and policy gradient methods require careful hyperparameter control to avoid suboptimal convergence. This phase is inherently difficult since the agent cannot distinguish walls from the box, making scores around -1980 expected. Most agents defaulted to spinning in place. The attachment bit was a promising idea worth pursuing further with DDQN.

3.3 Phase 2: World of Value-Based methods

All training used difficulty 2 with walls ON. A basic DDQN scored $-\mathbf{1981.80}$ [Submission 1 (649365) – Basic DDQN]. Biasing exploration toward FW actions improved this to $-\mathbf{1586.90}$ [Submission 2 (649324) – Forward Biased DDQN]. Adding PER and frame stacking scored $-\mathbf{1624.10}$ [Submission 3 (653458) – DDQN with PER and Stacking]; a D3QN variant scored $-\mathbf{1631.40}$ [Submission 5 (653526) – D3QN with PER and Stacking]. Retraining with 2000 max steps gave $-\mathbf{1613.30}$ [Submission 6 (655422) –

Forward Biased DDQN with 2000 maxsteps]. Revisiting the attachment bit with a GRU-based AttachNet plus forward biasing scored $-\mathbf{1968.00}$ [Submission 7 (653526) – Augmented DDQN with forward biasing]. Adding PER and clipping the wall penalty from -200 to -10 significantly improved the score to $-\mathbf{1204.70}$ [Submission 8 (655474) – Augmented DDQN with PER, stacking, and reward clipping].

Key takeaways: Forward biasing, PER, frame stacking, and reward clipping all contributed meaningful improvements. The agent has learned to move toward the box when detected, but still defaults to 360-degree spinning when all sensors are inactive – a behaviour I refer to as the *blind state*.

3.4 Phase 3: Modular Ideas

All training used difficulty 3 with walls ON. Retraining the best phase 2 agent scored $-\mathbf{791.68}$ [Submission 1 (656794) and 2 (657233) – Augmented DDQN with forward biasing, stacking, PER and reward clipping]. Adding blind-state reward shaping improved this to $-\mathbf{417.00}$ [Submission 3 (664873) – with shaping added]. Adding previous action as one-hot (forming a 24-bit observation) gave $-\mathbf{111.00}$ [Submission 4 (664113) – COMPASS]. Wall-scheduled training scored $-\mathbf{662.18}$ [Submission 5 (665351) – COMPASS with wall schedule]; adding a forward bonus scored $-\mathbf{1429.06}$ [Submission 6 (665357) – COMPASS with shaping]. Observing COMPASS was a strong pusher but weak finder, I introduced SPYDER (Finder-Pusher architecture), scoring $-\mathbf{188.50}$ [Submission 7 (664115) – SPYDER]. A 5-step TD variant scored $-\mathbf{1383.54}$ [Submission 8 (665360) – SPYDER with 5-step return].

Key takeaways: This phase was the easiest to train agents since the moving box eventually comes to the agent, requiring only a push to the boundary. COMPASS and SPYDER both successfully chased and pushed the box. However, both still struggle in the *blind state* when all sensors read zero.

3.5 Phase 4 / Test Phase: Endgame

Submitting phase 3 best agents directly gave $-\mathbf{1307.37}$ [Submission 1 (667142) – COMPASS] and $-\mathbf{1484.50}$ [Submission 2 (667130) – SPYDER]. The final model VEGATACH trained on difficulty 2 scored $-\mathbf{892.91}$ [Submission 3 (672131) – VEGATACH d2]. Curriculum training (600 eps diff 0 wall off, 1200 diff 0 wall on, 1400 diff 2 wall on, 1800 diff 3 wall on) improved this to $-\mathbf{763.63}$ [Submission 4 (675848) – VEGATACH curriculum]. Adding reward shaping scored $-\mathbf{614.37}$ [Submission 5 (676139) – VEGATACH shaped]. A variant with normalized blind streak observation scored $-\mathbf{1984.43}$ [Submission 6 (676216) – VEGATACH with blind streak].

3.6 Phase 3 Best Model in Detail - COMPASS and SPYDER

- **Algorithm Family:** COMPASS and SPYDER are built on the Double DQN algorithm with Prioritized Experience Replay (PER). Both use ϵ-greedy exploration with forward biasing, where the random action distribution favors forward movement to prevent the agent from learning to spin in place.

- **State Representation.** The state representation augments the raw 18-bit sensor observation with two hand-crafted features: an attachment bit predicted by a GRU-based network (AttachNet), and a one-hot encoding of the previous action. This forms a 24-dimensional observation per timestep. To provide temporal context in this POMDP setting, 4 consecutive augmented frames are stacked, yielding a 96-dimensional input to the Q-network. The attachment bit is the key hand-crafted feature – during training, it is inferred from the reward signal (+100 indicates attachment), while at inference, a trained GRU predicts it from the observation-action history.

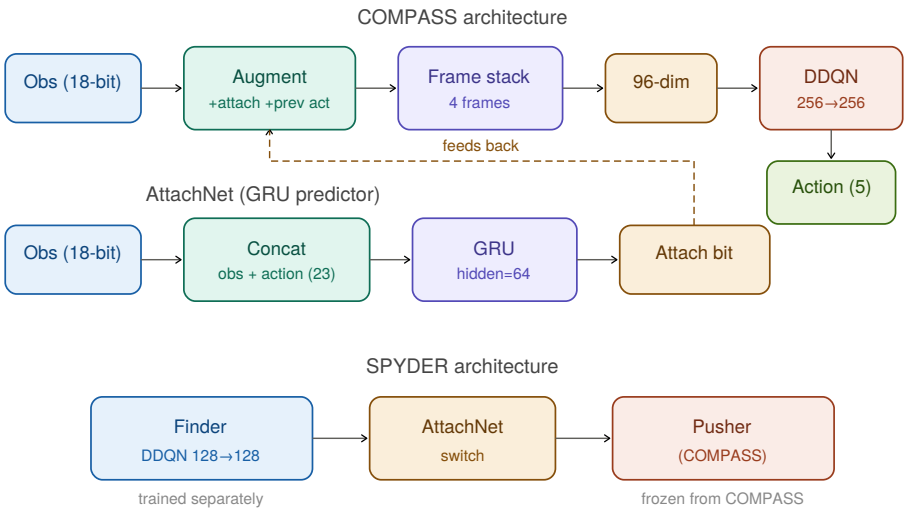

Figure 1: COMPASS and SPYDER Architecture

- **Training and Exploration Strategy:** COMPASS trains a single DDQN ($96 \to 256 \to 256 \to 5$) jointly with the AttachNet on difficulty 3. It was observed that COMPASS is a strong pusher but a weak finder—it struggles to locate the box when no sensors are active. This motivated SPYDER, a hierarchical Finder-Pusher architecture inspired by the submodule idea from the original OBELIX paper. SPYDER freezes the COMPASS Q-network and AttachNet, then trains a separate, smaller Finder network ($96 \to 128 \to 128 \to 5$) specifically for the box-finding sub-task on difficulty 2. At inference time, the AttachNet's prediction alternates between the Finder (when unattached) and the Pusher (when attached).

- **Reward Shaping:** Reward clipping (clamping the -200 wall penalty to -10) is applied during training to prevent catastrophic wall collisions from dominating the replay buffer and to allow the agent to learn from successful episodes. No additional reward shaping is used in either model.

- **Hierarchical / Behaviour-Based Ideas:** Both models are inspired heavily by the original Mahadevan & Connell(1991) paper's behavior-based architecture:
  - **Attachment bit as implicit state detection:** The original OBELIX paper uses distinct behavioral submodules (Find, Push, Unwedge) that activate based on internal state. COMPASS replicates this by augmenting observations with a predicted attachment bit, letting a single network implicitly learn different policies for find and push phases without explicit switching.
  - **Explicit Finder-Pusher decomposition:** SPYDER directly instantiates the paper's submodule philosophy — a dedicated Finder handles navigation while the Pusher (frozen COMPASS) handles box-pushing, with the AttachNet serving as the state-based arbiter that the original paper achieved through hard-coded transitions.

3.7 Final Best Model in Detail - VEGATACH Shaped

- **Algorithm Family:** The agent uses a *Recurrent Duelling Double DQN*. It combines Double DQN (to reduce Q-value overestimation), a Duelling architecture (separating state-value V from advantage A), a GRU recurrent layer (to handle partial observability by learning in the temporal domain), and episode-level prioritised replay with burn-in for stable recurrent training.

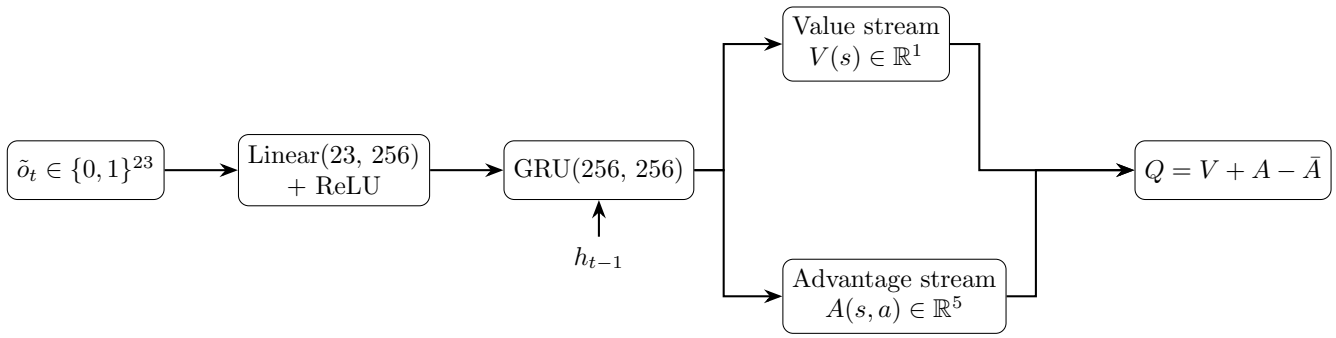

Figure 2: VEGATACH: Dueling Recurrent DDQN architecture.

The input layer maps the 23-dim observation to 256 units via ReLU. A single-layer GRU maintains hidden state $h_t \in \mathbb{R}^{256}$ across timesteps, enabling temporal memory. The output splits into a value stream $V(s)$ and an advantage stream $A(s,a)$, each with a 256-unit hidden layer, combined as $Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a')$.

- **State Representation:** The raw 18-bit observation (16 sonar bits + 1 IR bit + 1 stuck flag) is augmented with a one-hot encoding of the previous action $a_{t-1} \in \{0,\ldots,4\}$:

$$\tilde{o}_t = \big[\underbrace{o_t}_{\in\{0,1\}^{18}} \;\|\; \underbrace{\text{onehot}(a_{t-1})}_{\in\{0,1\}^{5}}\big] \in \{0,1\}^{23}$$

  The previous-action feature helps the recurrent network disambiguate states that look identical in sensor readings but were reached via different actions.

- **Exploration Strategy:** ϵ-greedy with linear decay from $1.0 \to 0.05$ over 2000 episodes. The random action distribution is context-aware:
  - If stuck ($o_t[17] = 1$): sample uniformly from turn-only actions {L45, L22, R22, R45}.
  - Otherwise: sample with forward bias — $P(\text{FW}) = 0.70$, turns share the remaining 0.30.

- **Reward Shaping:** Minor reward shaping was necessary to force exploration during *blind phases*when all the sensor bits are 0). Applied only *before attachment* and only in *blind states* (no sensor bits active, not stuck):
  - Moving forward while blind: +0.2 bonus (encourages exploration over spinning).
  - Consecutive blind turns: −0.1 after 5 in a row, −0.2 after 9 (penalises aimless rotation).
  - Streak resets on forward movement or exiting the blind state.
  - All rewards clipped at −10 to prevent the −200 stuck penalty from dominating gradients.

- **Hierarchical / Behaviour-Based Ideas:** The design draws on the original Mahadevan & Connell (1991) paper's behaviour-based architecture (Find → Push → Unwedge) in two ways:
  - **The reward shaping is *phase-aware*:** shaping applies only in the "Find" phase (before attachment) and is turned off during "Push," mirroring the paper's idea of distinct behavioral modules with separate reward signals.
  - **The exploration policy acts as a soft behavioral prior:** the stuck-detection branch forces turn-only actions (analogous to the Unwedge behavior), while the forward-biased default mimics the Find behavior's tendency to cover ground.

4 Experiments and Implementation Details

- **Training setup:** Most of my experiments were conducted previously using 5000 episodes with 1000 max steps, or lately 3000 episodes with 2000 max steps. For hardware, KD lab desktops were used for training and are equipped with 14th-gen i9 processors. Some algorithms were also trained on my personal MacBook Air M4 and a Linux laptop with an RTX 4060 and a Ryzen processor. On average, each algorithm took around 8 to 12 hours to train in my case.
- **Multi-task / curriculum learning:** Only the VEGATACH model was trained using curriculum learning; COMPASS and SPYDER were trained on difficulty levels 3 and 2, respectively.
- **Local evaluation strategy:** Every 50 or 100 training episodes, the agent is evaluated greedily (no exploration) over 10 episodes per setting using fixed seeds from 0–9.
- **Hyperparameter tables:**

Table 1: COMPASS hyperparameters.

Category	Param	Value
Q-Net	Arch.	$96 \to 256 \to 256 \to 5$
	Frame stack	4
	Input dim	$24 \times 4 = 96$
Optim	LR	1×10^{-4}
	Batch	256
	γ	0.99
	Loss	Smooth L1
Target	τ (Polyak)	0.005
Explore	ϵ	$1.0 \to 0.02$ / 1.5M
PER	Capacity	500K
	α / β	0.6 / $0.4 \to 1.0$
	Warmup	5K
Attach. Pred.	Arch.	GRU($23 \to 64$)
	LR	5×10^{-4}
	Freq / Epochs	10 eps / 4
	Traj. buf.	500 eps
Train	Episodes	3000
	Difficulty	3
	Reward clip	-10

Table 2: SPYDER hyperparameters.

Category	Param	Value
Find Q-Net	Arch.	$96 \to 128 \to 128 \to 5$
	Frame stack	4
	Input dim	$24 \times 4 = 96$
Optim	LR	1×10^{-4}
	Batch	128
	γ	0.99
	Loss	Smooth L1
Target	τ (Polyak)	0.005
Explore	ϵ	$1.0 \to 0.02$ / 1.5M
PER	Capacity	500K
	α / β	0.6 / $0.4 \to 1.0$
	Warmup	5K
Frozen Modules	Push Q-Net	from COMPASS
	Predictor	from COMPASS
	Push arch.	$96 \to 256 \to 256 \to 5$
Train	Episodes	2000
	Difficulty	2 (for finder)
	Reward clip	-10

Table 3: VEGATACH hyperparameters.

Category	Param	Value
Dueling R-DDQN	Input / FC	$23 \to 256$
	GRU	$256 \to 256$
	Val Stream / Adv Stream	$256 \to 256 \to 1$ / $256 \to 256 \to 5$
Optim	LR / Batch	1×10^{-4} / 32 sequences
	γ / Loss	0.99 / MSE
Target	Hard sync	every 500 updates
Explore	ϵ	$1.0 \to 0.05$ / 2000 episodes
Episode Replay	Capacity / Seq / Burn-in	2000 / 40 / 20
	α / β_0	0.6 / 0.4
Reward Shaping	FW (blind)	$+0.2$
	Turn ≥ 5 / ≥ 9	-0.1 / -0.2
	Reward clip(negative side)	-10
Train	Episodes	5000
	Updates/episode	8
Curriculum Schedule	Ep 1–600	Diff 0, wall off
	Ep 601–1800	Diff 0, wall on
	Ep 1801–3200	Diff 2, wall on
	Ep 3201–5000	Diff 3, wall on

5 Results

Figure 3: VEGATACH (shaped) learning curves.

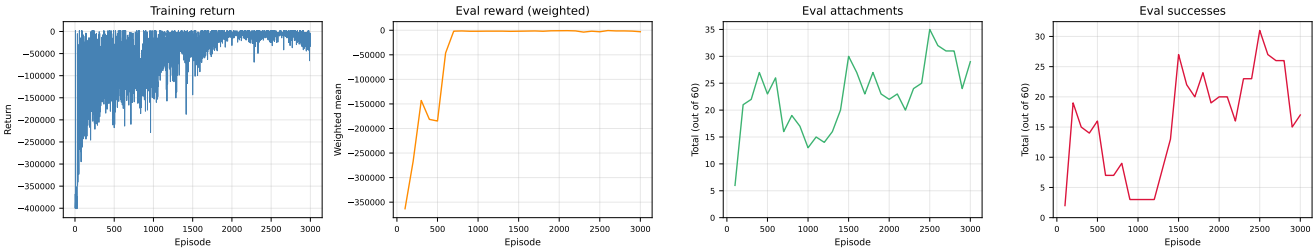

Table 4: Model performance across development and test phases.

Model	**Phase 1** (Diff 0)	**Phase 2** (Diff 2)	**Phase 3** (Diff 3)	**Phase 4** (Test, All)
Sample Submission	−17128.10	–	–	–
Default DDQN	−379036.50	–	–	–
Default DDQN (more episodes)	−215282.10	–	–	–
DDQN with LSTM	−29248.90	–	–	–
Basic D3QN	−102740.80	–	–	–
Q-Lambda	−1997.40	–	–	–
DQN (aggressive exploration)	−1989.30	–	–	–
Augmented DDQN	**−1972.70**	–	–	–
Basic PPO	−1980.70	–	–	–
Basic DDQN	–	−1981.80	–	–
Forward Biased DDQN	–	−1586.90	–	–
DDQN with PER and Stacking	–	−1624.10	–	–
D3QN with PER and Stacking	–	−1631.40	–	–
Forward Biased DDQN (2000 steps)	–	−1613.30	–	–
Augmented DDQN (forward biasing)	–	−1968.00	–	–
Augmented DDQN + PER + clipping	–	**−1204.70**	–	–
Augmented DDQN (retrained)	–	–	−791.68	–
+ reward shaping	–	–	−417.00	–
COMPASS	–	–	**−111.00**	−1307.37
COMPASS (wall schedule)	–	–	−662.18	–
COMPASS (shaping)	–	–	−1429.06	–
SPYDER	–	–	−188.50	−1484.50
SPYDER (5-step return)	–	–	−1383.54	–
VEGATACH (diff 2)	–	–	–	−892.91
VEGATACH (curriculum)	–	–	–	−763.63
VEGATACH (shaped)	–	–	–	**−614.37**
VEGATACH (blind streak)	–	–	–	−1984.43

Table 5: Test phase: mean ± std cumulative reward per setting.

	Wall ON			**Wall OFF**			
Model	Diff 0	Diff 2	Diff 3	Diff 0	Diff 2	Diff 3	**Weighted**
VEGATACH (shaped)	-988.9 ± 2097.9	-586.9 ± 2249.4	-1052.5 ± 2573.3	-1523.9 ± 2044.1	-409.4 ± 1921.9	1268.0 ± 1211.0	**−614.37**
VEGATACH (curriculum)	-1588.3 ± 1960.4	-1324.4 ± 1836.7	-1580.6 ± 2844.4	-381.1 ± 1981.1	-393.0 ± 1966.8	1786.8 ± 253.5	−763.63
VEGATACH (diff 2)	-1455.1 ± 1865.1	-2128.2 ± 2098.7	-494.9 ± 1997.3	-1616.8 ± 1115.0	-603.1 ± 1733.2	1640.4 ± 418.9	−892.91
COMPASS	-1977.2 ± 8.4	-2120.1 ± 1931.5	-1230.3 ± 2816.9	-1988.5 ± 12.3	-1392.7 ± 1174.8	1567.3 ± 497.4	−1307.37
SPYDER	-2234.8 ± 777.8	-2223.4 ± 461.6	-995.9 ± 1799.8	-1988.4 ± 12.9	-1986.8 ± 13.3	1022.6 ± 1475.9	−1484.50

Out of all the models, I feel the VEGATACH family of models have done consistently well across all difficulty levels. The reason for this was that they showed very stable learning during training, as evident from the curves above, unlike the DDQN-based approaches, and their scores were also decent.

6 Error Analysis and Discussion

- **Failure modes:** At difficulty 0 and 2, failures stem from the *blind state* – when no sensors detect the box, all agents default to spinning rather than exploring forward. At difficulty 2, blinking causes the agent to lose track mid-approach. At difficulty 3, the agent cannot predict the box trajectory and relies on the box entering the sensor range. With walls, the agent frequently pushes the box into the wall and gets stuck due to perceptual aliasing (the wall and the box produce identical sensor readings).
- **Limitations and future work:** The biggest limitation is the blind state – a learned exploration policy (e.g. systematic sweeping) or a combination of full exploration when blind and trained policy otherwise would significantly improve performance on difficulty 0 and 2.

- **Surprising findings:** Reward clipping ($-200 \rightarrow -10$) had a disproportionately large impact – without it, wall collisions dominated the replay buffer and prevented learning useful behavior. The same is the case with Forward Biasing – because out of 5 actions, only 1 is a movement action, so it made sense to have more forward actions in the replay buffer to allow the agent to learn from it. Agents trained with wall ON automatically generalized to wall OFF environments, making wall-scheduling-based training pointless

- **Key achievements:** Starting from basic textbook-level algorithms, I progressively developed three agents – COMPASS, SPYDER, and VEGATACH – which performed consistently well across multiple difficulty levels. Key techniques that drove improvement were observation augmentation (attachment bit, previous action), prioritized replay, reward clipping, forward-biased exploration, and curriculum training.
- **Lessons learned:** Specialized models (COMPASS, SPYDER) excel on individual difficulties but fail to generalize – a single recurrent architecture (VEGATACH) with curriculum training generalizes better. Finally, visualizing agent behavior was invaluable – the blind state and weak-finder problems were only discovered through visual inspection, not from reward curves alone. All in all, I understood that POMDP problems are extremely difficult to tackle, and basic textbook RL algorithms cannot solve them on their own without careful modifications.
- **Takeaways from RL in POMDPs:** Partial observability is the central challenge – the agent cannot distinguish walls from boxes and has no information when sensors are inactive. Augmenting observations with inferred state (attachment prediction) and temporal context (frame stacking or recurrent memory) is essential.
- **Future directions:** A learned exploration policy for the blind state (e.g. systematic sweeping instead of spinning) would yield the largest gains. Longer recurrent context or transformer-based memory could better handle wall-box aliasing. Extending to multi-box environments, realistic push physics, and sim-to-real transfer are natural next steps.

LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report? ChatGPT
- In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	No	
Ideation / brainstorming	No	
Debugging small code snippets	Yes	Most of the code structure is similar to the provided DDQN code file, however, minor implementation bugs were solved by LLM
Plotting / visualization code	Yes	LLM gave a separate plotting code which used my train and eval logs to create the plots
Grammar / writing style	Yes	Asked LLM for help with attaching images, resizing tables
Report structure / section ideas	No	
Diagram	Yes	LLM was used to generate the network diagram to be used in Latex

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
  If yes, please mention clearly (papers, blogs, GitHub repos, textbooks etc.)
  Yes. The 2 reference books the instructor suggested to understand the theory, the OBELIX paper provided in the capstone document.
- Apart from any LLM usage, what was your own contribution?
  Briefly describe your personal understanding, reasoning, implementation effort, analysis, and experimentation.
  All experimental ideas, codes, and implementation choices are of my own. LLM was used only to generate the plots, implement minor bug fixes, and assist with LaTeX issues during report writing.

References

[1] S. Mahadevan and J. Connell, "Automatic programming of behavior-based robots using reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 1991.